

MANUAL DE ADMINISTRADOR

Juez Logit

Sistema Inteligente de Arbitraje para Combates de Robótica

Versión del documento: 1.0

Fecha: Julio 2026

Audiencia: Administradores / DevOps del sistema

1. Introducción

Este manual está dirigido a la persona responsable de instalar, configurar, desplegar y mantener el sistema Juez Logit: un sistema de arbitraje automatizado para combates de robótica (categoría antweight) que combina una Raspberry Pi con cámara, un backend en FastAPI y el modelo de inteligencia artificial Gemini para determinar el ganador de cada combate.

El documento cubre la arquitectura del sistema, los requisitos de instalación, la configuración de variables de entorno y base de datos, el ciclo de vida operativo (arranque, grabación, análisis), la administración de datos, y una sección de seguridad con hallazgos que deben corregirse antes de un despliegue en producción.

Hallazgo crítico de seguridad detectado en el código fuente

El archivo backend/api_keys.py contiene una clave de API de Gemini escrita directamente en texto plano dentro del código.

El .gitignore del proyecto excluye un archivo llamado ".api_keys.py" (con un punto al inicio), pero el archivo real se llama "api_keys.py" (sin punto). Por esta discrepancia, el archivo NO está siendo ignorado por Git y la clave puede haber quedado expuesta en el historial del repositorio.

Acción recomendada inmediata: revocar/rotar esta clave en Google AI Studio, mover la clave a una variable de entorno (.env) y purgarla del historial de Git. Ver sección 9 (Seguridad) para el procedimiento completo.

2. Arquitectura del sistema

El sistema está compuesto por tres bloques principales:

- Cliente Raspberry Pi (`raspberry_client.py`): controla la cámara USB mediante `ffmpeg`, graba el combate en segmentos, los une en un único archivo de video y lo sube al backend.
- Backend (FastAPI): recibe el video y las imágenes de referencia, coordina la grabación remota, invoca al modelo de IA (Gemini) para el análisis, procesa la respuesta, guarda resultados en PostgreSQL y sirve la interfaz web.
- Motor de IA (Gemini API): analiza el video del combate junto con imágenes de referencia "antes/después" de cada robot, y devuelve un veredicto estructurado según un criterio de puntuación técnica.

Flujo típico de un combate:

1. El administrador inicia la grabación desde la interfaz web (endpoint `/start_recording`).
2. El cliente Raspberry Pi detecta el cambio de estado y comienza a grabar segmentos de video con `ffmpeg`.
3. Al finalizar (`/stop_recording`), la Raspberry une los segmentos y sube el video final al backend (`/upload`).
4. El administrador o el juez ejecuta el análisis (`/analyze`), adjuntando opcionalmente fotos de "antes/después" de cada robot.
5. El backend construye un prompt técnico y envía el video + imágenes al modelo Gemini.
6. La respuesta de Gemini se interpreta (`parser_service.py`) y se guarda en la base de datos (tabla `analisis`).
7. El resultado (ganador y puntuaciones) se muestra en la interfaz web.

Ver también los diagramas incluidos en el repositorio: `assets/Arquitectura.png`, `assets/ui-main.png` y `assets/ui-results.png`.

3. Requisitos previos

3.1 Hardware

- Raspberry Pi Zero 2W (o equivalente) con cámara USB compatible con Video4Linux2 (v4l2).
- Servidor o equipo donde correrá el backend (puede ser el mismo u otro en la misma red / en la nube).
- Red local o conexión que permita comunicación HTTP entre la Raspberry y el backend (puertos 8000).

3.2 Software

Componente	Versión / Requisito
Python	3.12.8 (definido en backend/.python-version)
FastAPI	0.136.1
Uvicorn	0.46.0
PostgreSQL	Servidor accesible (local o gestionado, ej. Render)
ffmpeg	Instalado en la Raspberry Pi (usado por raspberry_client.py)
Node.js	Solo si se usa server.js (actualmente vacío / no utilizado)

Las dependencias exactas de Python están listadas en backend/requirements.txt e incluyen: fastapi, uvicorn (implícito), httpx, Jinja2, psycpg2-binary, pydantic, python-dotenv, python-multipart, requests, starlette, entre otras.

4. Instalación y despliegue

4.1 Obtener el código

```
git clone https://github.com/PatrickZ29/Nhrl.git
cd Nhrl/backend
```

4.2 Crear entorno virtual e instalar dependencias

```
python -m venv venv
# Linux / macOS
source venv/bin/activate
# Windows
venv\Scripts\activate

pip install -r requirements.txt
```

4.3 Configurar variables de entorno

Crear un archivo `.env` dentro de la carpeta `backend/` (este archivo ya está excluido correctamente en `.gitignore`) con el siguiente contenido de referencia:

```
# Carpeta donde se guardan los videos subidos
VIDEO_FOLDER=./videos

# Modelo de Gemini a utilizar
GEMINI_MODEL=gemini-2.5-flash

# Clave de API de Gemini (NO dejarla en el código fuente)
GEMINI_API_KEY=coloque_aqui_su_clave_nueva

# Conexión a base de datos: opción A (URL completa, recomendada en la nube)
DATABASE_URL=postgresql://usuario:password@host:5432/robot_ai

# Conexión a base de datos: opción B (variables sueltas, uso local)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=robot_ai
DB_USER=postgres
DB_PASSWORD=defina_una_contrasena_segura
```

Nota importante: actualmente `backend/api_keys.py` define la clave de forma fija (hardcodeada) y no lee `GEMINI_API_KEY` del entorno. Antes de desplegar, se debe modificar ese archivo para leer la clave con `os.getenv("GEMINI_API_KEY")` — ver sección 9.2 para el detalle.

4.4 Inicializar y ejecutar el backend

La base de datos se inicializa automáticamente al arrancar la aplicación (evento `startup` en `main.py`), creando la tabla `analisis` si no existe.

```
cd backend
uvicorn main:app --reload
```

Por defecto la aplicación queda disponible en: `http://localhost:8000`

Endpoint de verificación de salud: `GET /health` → `{"status": "ok"}`

4.5 Configurar el cliente Raspberry Pi

El script `backend/robot/raspberry_client.py` debe copiarse y ejecutarse directamente en la Raspberry Pi (requiere `ffmpeg` instalado y acceso al dispositivo de cámara, por defecto `/dev/video0`).

Antes de ejecutarlo, editar la lista `SERVERS` al inicio del archivo con las direcciones IP reales donde corre el backend en la red local:

```
SERVERS = [  
    "http://<ip_del_backend_1>:8000",  
    "http://<ip_del_backend_2>:8000",  
]
```

El script consulta periódicamente `/recording_status` en el primer servidor de la lista que responda, e inicia/pausa/detiene la grabación local en consecuencia. Al detener la grabación, une los segmentos con `ffmpeg` y sube el archivo resultante mediante `POST /upload`.

```
python raspberry_client.py
```

4.6 Despliegue en la nube (Render)

El repositorio incluye un archivo `render.yaml` para desplegar en Render, aunque actualmente está vacío. Al configurar el despliegue, el administrador debe definir manualmente en el panel de Render (o completar `render.yaml`):

- Build command: `pip install -r backend/requirements.txt`
- Start command: `uvicorn main:app --host 0.0.0.0 --port $PORT` (ejecutado desde la carpeta `backend`)
- Variables de entorno: las mismas descritas en la sección 4.3 (`GEMINI_MODEL`, `GEMINI_API_KEY`, `DATABASE_URL`, etc.), configuradas como "Secret" en el panel de Render.
- Una base de datos PostgreSQL gestionada por Render (o externa) enlazada mediante `DATABASE_URL`.

5. Configuración del sistema (backend/config.py)

Todas las variables de configuración del backend se centralizan en config.py y se cargan mediante python-dotenv.

Variable	Valor por defecto	Descripción
VIDEO_FOLDER	backend/videos	Carpeta donde se almacenan los videos de combates subidos
GEMINI_MODEL	gemini-2.5-flash	Modelo de Gemini configurado por defecto (nota: gemini_service.py usa actualmente un modelo distinto codificado directamente, ver sección 8.2)
DATABASE_URL	(vacío)	Cadena de conexión completa a PostgreSQL; si está definida, tiene prioridad
DB_HOST	localhost	Host de PostgreSQL (usado solo si no hay DATABASE_URL)
DB_PORT	5432	Puerto de PostgreSQL
DB_NAME	robot_ai	Nombre de la base de datos
DB_USER	postgres	Usuario de la base de datos
DB_PASSWORD	1234	Contraseña (¡cambiar este valor por defecto en cualquier entorno real!)

El administrador debe asegurarse de sobrescribir estos valores por defecto (especialmente DB_PASSWORD) mediante el archivo .env en cada entorno.

6. Operación diaria del sistema

6.1 Ciclo de grabación y análisis

8. Abrir la interfaz web en `http://<host_backend>:8000`.
9. Iniciar la grabación (`GET /start_recording`); el estado puede consultarse en `/recording_status`.
10. Durante el combate, es posible pausar (`/pause_recording`) y reanudar (`/resume_recording`) la grabación si es necesario.
11. Detener la grabación (`/stop_recording`); la Raspberry unirá los segmentos y subirá el video automáticamente vía `/upload`.
12. Confirmar que el video llegó correctamente con `GET /check_status`.
13. Ejecutar el análisis (`POST /analyze`) indicando el nombre de cada robot y, opcionalmente, adjuntando hasta 4 imágenes (inicio/final de cada robot). Si no se sube un video nuevo, el sistema usa automáticamente el último video disponible en la carpeta de videos.
14. Revisar el resultado mostrado en pantalla: identificación de cada robot, piezas perdidas, resumen del combate, ganador y puntuación detallada (Agresividad, Condición, Daño, Control).

6.2 Consultar historial

El endpoint `GET /historial` devuelve los últimos 50 análisis guardados en la base de datos, ordenados del más reciente al más antiguo, incluyendo robots enfrentados, ganador, tiempo de procesamiento y fecha.

7. Referencia de endpoints de la API

Endpoint	Método	Descripción
/	GET	Interfaz web principal (index.html)
/health	GET	Verificación de estado del servicio
/check_status	GET	Consulta si hay un video listo para analizar
/start_recording	GET	Inicia la grabación remota
/pause_recording	GET	Pausa la grabación en curso
/resume_recording	GET	Reanuda una grabación pausada
/stop_recording	GET	Detiene la grabación (dispara la subida del video)
/recording_status	GET	Estado actual de grabación (recording / paused)
/upload	POST	Recibe el archivo de video desde la Raspberry Pi
/analyze	POST	Ejecuta el análisis del combate con IA y guarda el resultado
/historial	GET	Devuelve los últimos 50 análisis registrados

Nota de administración: ninguno de estos endpoints tiene autenticación actualmente (ver sección 9.3). En un entorno expuesto a Internet, cualquier persona con la URL podría iniciar/detener grabaciones o disparar análisis.

8. Administración del motor de IA (Gemini)

8.1 Prompt de evaluación

El criterio de arbitraje está definido íntegramente en `backend/services/prompt_service.py` mediante una plantilla de instrucciones muy detallada que exige a Gemini:

- Excluir de la evaluación a un robot "Juez Bot" (cuadrado, gris metálico) que actúa como cámara/observador.
- Mantener la identidad de cada robot por sus características físicas, no por su posición en pantalla.
- Puntuar cuatro criterios: Agresividad (0-15), Condición (0-5), Daño infligido (0-10) y Control (0-10), para un máximo de 40 puntos por robot.
- Declarar ganador por mayor puntuación total, con reglas estrictas de desempate, evitando declarar "empate" salvo que el combate haya sido prácticamente nulo.

Si el administrador necesita ajustar el criterio de arbitraje (por ejemplo, cambiar los rangos de puntuación o agregar nuevas categorías de robot), debe modificar el texto de esta plantilla en `prompt_service.py`. Cualquier cambio debe probarse con videos de referencia antes de usarse en un evento real, ya que el formato de salida esperado por `parser_service.py` depende de que Gemini respete exactamente la estructura solicitada.

8.2 Modelo utilizado y costos

Se detectó una inconsistencia de configuración: `config.py` define `GEMINI_MODEL="gemini-2.5-flash"` como variable de entorno, pero `gemini_service.py` no utiliza esa variable — en su lugar tiene el modelo escrito directamente en el código como `"gemini-3.1-flash-lite"`. El administrador debe decidir cuál es el comportamiento deseado y unificarlo, idealmente haciendo que `gemini_service.py` importe y use `GEMINI_MODEL` desde `config.py` en lugar de un valor fijo.

Cada llamada a `/analyze` envía el video completo codificado en base64 a la API de Gemini, lo cual consume cuota/costo por cada análisis. Se recomienda monitorear el uso desde el panel de Google AI Studio / Google Cloud, especialmente si el sistema se usa en eventos con muchos combates.

8.3 Interpretación de resultados (`parser_service.py`)

La respuesta de texto de Gemini se interpreta mediante expresiones regulares que buscan literalmente las etiquetas GANADOR, Agresividad, Condición, Daño y Control asociadas al nombre de cada robot. Si Gemini no respeta el formato exacto solicitado en el prompt (por ejemplo, por un cambio de modelo o una actualización de la API), el parser devolverá guiones ("-") en los campos no encontrados. Ante resultados vacíos recurrentes, el administrador debe revisar primero el texto crudo devuelto por Gemini antes de sospechar de un error de base de datos.

9. Seguridad

9.1 Resumen de hallazgos

Hallazgo	Riesgo	Recomendación
Clave de API de Gemini escrita en texto plano en backend/api_keys.py	Alto	Revocar/rotar la clave y moverla a variable de entorno
Discrepancia en .gitignore: excluye ".api_keys.py" pero el archivo real es "api_keys.py"	Alto	Corregir el .gitignore y purgar el archivo del historial de Git
Contraseña de base de datos por defecto "1234" en config.py	Medio	Definir DB_PASSWORD real mediante variable de entorno en todo entorno no local
Endpoints de la API sin autenticación	Medio	Añadir autenticación (API key, token o red privada) antes de exponer el backend a Internet
Video y datos de combates sin cifrado en tránsito si no se usa HTTPS	Medio	Servir el backend detrás de HTTPS (por ejemplo, mediante Render o un proxy inverso con TLS)

9.2 Procedimiento recomendado para corregir la clave expuesta

15. Ir a Google AI Studio y revocar/eliminar la clave de API actual.
16. Generar una nueva clave de API de Gemini.
17. Modificar backend/api_keys.py para leer la clave desde una variable de entorno, por ejemplo:

```
import os

def get_api_key():
    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        raise Exception("API KEY no configurada")
    return api_key
```

18. Agregar GEMINI_API_KEY al archivo .env local y a las variables de entorno del entorno de despliegue (Render u otro).
19. Corregir el .gitignore para que la exclusión coincida exactamente con el nombre real del archivo, si se decide mantener alguna variante local de claves fuera del control de versiones.
20. Purgar el archivo del historial de Git (por ejemplo, con git filter-repo o BFG Repo-Cleaner) si el repositorio es o fue público, ya que revertir el commit no elimina la clave del historial.

9.3 Autenticación y exposición de red

Actualmente cualquier persona con acceso a la URL del backend puede iniciar o detener grabaciones, subir videos o disparar análisis, ya que ningún endpoint valida identidad. Si el sistema se va a exponer fuera de

una red local controlada (por ejemplo, en un torneo con público o en la nube), se recomienda añadir al menos un mecanismo simple de autenticación (cabecera con token compartido) en los endpoints de control (/start_recording, /stop_recording, /analyze, etc.).

10. Mantenimiento

10.1 Gestión de la carpeta de videos

Los videos subidos se acumulan en VIDEO_FOLDER (por defecto backend/videos) con nombres del tipo fight_AAAAMMDD_HHMMSS.mp4. Esta carpeta no se limpia automáticamente. El administrador debe definir una política de retención (por ejemplo, archivar o eliminar videos de eventos ya finalizados) para evitar que el disco se llene, especialmente en dispositivos con almacenamiento limitado.

10.2 Respaldo de la base de datos

La tabla analisis contiene el historial completo de combates evaluados. Se recomienda programar respaldos periódicos de PostgreSQL (pg_dump) antes de cada evento importante y conservar copias fuera del servidor de producción.

```
pg_dump -U <usuario> -h <host> -d robot_ai -f backup_robot_ai.sql
```

10.3 Actualización de dependencias

Antes de actualizar versiones en requirements.txt (FastAPI, Starlette, Pydantic, etc.), probar en un entorno de staging, ya que actualizaciones mayores de FastAPI/Starlette pueden requerir cambios de código. Verificar siempre que el evento startup de inicialización de base de datos siga funcionando tras cualquier actualización.

10.4 Archivos sin uso actual

server.js y render.yaml están presentes en el repositorio pero actualmente vacíos. El administrador debe confirmar si tienen un uso planeado (por ejemplo, un frontend adicional en Node.js o una configuración de despliegue automatizado en Render) o si pueden eliminarse para simplificar el mantenimiento del repositorio.

11. Solución de problemas

Síntoma	Posible causa	Acción sugerida
El servidor imprime "Aviso: no se pudo inicializar la base de datos" al arrancar	Credenciales o conexión de PostgreSQL incorrectas	Revisar DATABASE_URL o las variables DB_* en el .env; verificar que el servidor de base de datos esté accesible
La Raspberry Pi muestra "No hay servidor disponible"	Ninguna IP de la lista SERVERS responde /recording_status	Verificar la conectividad de red y que el backend esté corriendo en alguna de las IPs configuradas
/analyze devuelve "Error al procesar el video con Gemini"	Clave de API inválida, cuota agotada o video corrupto	Revisar la clave de API, la cuota en Google AI Studio y que el archivo de video no esté vacío o dañado
El ganador o los puntajes aparecen como "-"	Gemini no respondió con el formato exacto esperado por el parser	Revisar el texto crudo ("response") devuelto y comparar con el formato exigido en prompt_service.py
"No hay videos disponibles" al analizar sin subir video	La carpeta VIDEO_FOLDER está vacía	Confirmar que el video se haya subido correctamente vía /upload antes de analizar

12. Estructura del proyecto (referencia rápida)

Programa robot funcional2/	
— assets/	Imágenes de documentación (UI, arquitectura)
— backend/	
— main.py	Punto de entrada FastAPI
— config.py	Variables de configuración (.env)
— database.py	Conexión y consultas a PostgreSQL
— api_keys.py	Clave de Gemini (revisar sección 9)
— models/analysis_model.py	Modelo de datos de un análisis
— routers/analyze_router.py	Endpoints de grabación, subida y análisis
— services/gemini_service.py	Llamada a la API de Gemini
— services/prompt_service.py	Plantilla de criterios de arbitraje
— services/parser_service.py	Interpretación de la respuesta de Gemini
— services/video_service.py	Utilidad de guardado de video
— robot/raspberry_client.py	Script a ejecutar en la Raspberry Pi
— templates/index.html	Interfaz web
— videos/	Videos subidos (no versionado)
— requirements.txt	Dependencias de Python
— rules_engine.py	Utilidad auxiliar de parseo JSON
— server.js	Sin uso actual (vacío)
— render.yaml	Sin uso actual (vacío)
— README.md	Documentación funcional del proyecto

13. Anexo: variables de entorno completas

Variable	Obligatoria	Ejemplo
GEMINI_API_KEY	Sí (tras corregir api_keys.py)	Alza...
GEMINI_MODEL	No (valor por defecto: gemini-2.5-flash)	gemini-2.5-flash
VIDEO_FOLDER	No (valor por defecto: backend/videos)	/data/videos
DATABASE_URL	Recomendada en la nube	postgresql://user:pass@host:5432/robot_ai
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Alternativa a DATABASE_URL (uso local)	localhost / 5432 / robot_ai / postgres / ****

Fin del manual de administrador.